

What the code does is it operates to do a series of functions:

Menu list:

For this it lists the menu for the functions, so it lists ARP, Speedtest, Port scanning, sniff test, and a 'quit' option to exit the program.

It takes your input as an integer value and it goes into the specified function that is tied to the value.

ARP:

This function commits an ARP scan with a specified ip address.

```
Enter target IP or range for ARP scan: 10.135.123.141
IP                               MAC Address
-----
10.135.123.141                  fc:77:74:6a:05:e0
```

It will then display the address along with a mac address tied to it.

Speedtest:

Using a speedtest library I added it to the program since it could be helpful to identify if a network issue is based off the network itself or the device.

```
# Speedtest
def check_bandwidth():
    try:
        st = speedtest.Speedtest()
        download_speed = st.download() / 1_000_000
        upload_speed = st.upload() / 1_000_000
        ping = st.results.ping

        print(f"Download Speed: {download_speed:.2f} Mbps")
        print(f"Upload Speed: {upload_speed:.2f} Mbps")
        print(f"Ping Latency: {ping:.2f} ms")
    except Exception as e:
        print(f"Error during speed test: {e}")
```

It takes a second and then displays the download, upload, and latency of the device's connection to the network.

Port Scanning:

This one took a bit and also takes a fair amount to get going since it requires a scan of every port, once I timed it I saw that it takes about 18 minutes or so for it to finish every port scan. It also displays the open ports as they are being found.

```
Port Scan
def scan_port(target_ip, port):
    try:
        sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        sock.settimeout(1)
        result = sock.connect_ex((target_ip, port))
        if result == 0:
            print(f"Port {port}: Open")
        sock.close()
    except socket.error as e:
        print(f"Error on port {port}: {e}")

def scan_host(target):
    try:
        target_ip = socket.gethostbyname(target)
        print("-" * 50)
        print(f"Scanning target: {target_ip}")
        print(f"Scan started at: {datetime.now()}")
        print("-" * 50)

        for port in range(1, 1024):
            scan_port(target_ip, port)

    except socket.gaierror:
        print("Hostname could not be resolved.")
    except socket.error as e:
        print(f"Socket error: {e}")
```

The way it works is by checking if the port allows the device to make a connection, it then returns a value that shows it as open or close, and if it is open it displays it on the results page. Were there to be an issue with the connection, then it would return an error message. There were definitely better alternatives to this, probably, but I invested too much into this to work.

The sniff test:

The snifftest is used a lot of time to get as well.

```
def block_ip(ip_address):
    system_os = platform.system()

    try:
        if system_os == "Windows":
            print(f"[!] Blocking IP (Windows): {ip_address}")
            subprocess.run(["netsh", "advfirewall", "firewall", "add", "rule",
                            f"name=Block_{ip_address}", "dir=in", "action=block",
                            f"remoteip={ip_address}"], check=True)
        elif system_os == "Linux":
            print(f"[!] Blocking IP (Linux): {ip_address}")
            subprocess.run(["iptables", "-A", "INPUT", "-s", ip_address, "-j", "DROP"], check=True)
        else:
            print(f"[!] Blocking not supported on {system_os}")
    except subprocess.CalledProcessError:
        print(f"[-] Failed to block {ip_address}")
```

Starting with the ip blocking, creating a blacklist that will lock in any ip addresses that appear suspicious by a few criteria that will follow based off different functions.

```
# Logs (if the code wants to -_-) and Packet sniffing
captured_packets = []
connection_counter = defaultdict(int)
ip_blacklist = set()
start_time = time.time()

def analyze_packet(packet):
    global captured_packets
    captured_packets.append(packet)

    if IP in packet:
        src = packet[IP].src
        dst = packet[IP].dst
        connection_counter[src] += 1

        if time.time() - start_time < 10 and connection_counter[src] > 100:
            if src not in ip_blacklist:
                print(f"[!] High traffic volume from {src}")
                ip_blacklist.add(src)
                if hasattr(args, 'block') and args.block: # Check if args has "block" attribute to prevent the "restart" message :(
                    block_ip(src)

    if TCP in packet:
        dport = packet[TCP].dport
        if dport in [23, 21]:
            print(f"[!] Suspicious TCP port from {src} -> {dport}")
            if hasattr(args, 'block') and args.block and src not in ip_blacklist: # Check if args has "block" attribute, same as earlier
                ip_blacklist.add(src)
                block_ip(src)

    elif UDP in packet:
        print(f"[!] UDP packet: {src} -> {dst}")
    elif ICMP in packet:
        print(f"[!] ICMP packet: {src} -> {dst}")
    else:
        print(f"[!] Unknown protocol from {src}")
```

This one initiates the process, and it also shows how the blacklist can start to function by finding weird tcp packs or high volume from a source and then adding them to the blacklist which will block them.

```
#look more into this packet sniffing as it's not saving the pcap file when pressing ctrl+ c
def start_sniffing(interface=None, pcap_file="captured_traffic.pcap"):
    print("[*] Sniffing started. Press Ctrl+C in the IDLE shell to stop...")
    try:
        sniff(filter="ip", prn=analyze_packet, iface=interface, store=False)
    except KeyboardInterrupt:
        print(f"\n[*] Sniffing stopped. Saving packets to {pcap_file}") #This does not seem to work so I'm not sure how to make it work
        wrpcap(pcap_file, captured_packets)
        print("[*] PCAP saved successfully.")
    except PermissionError:
        print("[!] Permission denied >:). Run IDLE as administrator for sniffing.")
```

On this part it is supposed to collect the data stream into a pcap file, however it has come to me that the reason it doesn't work is because when I cancel the process, it abruptly ends the process, meaning it doesn't save anything, I could have implemented a way to not abruptly stop the process, meaning it would have stopped the process, saved the file, and then shut down the process, looping back to the menu list.

```
if __name__ == "__main__":
    while True:
        mode_choice = get_arguments_idle()

        if mode_choice == "1":
            args = type('Args', (), {})()
            args.target = input("Enter target IP for ARP scan: ")
            if args.target:
                result = arp_scan(args.target)
                print_arp_results(result)
            else:
                print("[!] Target IP or range can't be empty.")
        elif mode_choice == "2":
            args = type('Args', (), {})()
            args.target = input("Enter target IP or hostname for port scan: ")
            if args.target:
                scan_host(args.target)
            else:
                print("[!] Target IP or hostname can't be empty.")
        elif mode_choice == "3":
            check_bandwidth()
        elif mode_choice == "4":
            args = type('Args', (), {})() # Create a simple object for Sniffing 'sniff sniff'
            args.interface = input("Enter network interface for sniffing (leave blank for all): ") or None
            args.pcap = input(f"Enter filename to save captured packets (default: captured_traffic.pcap): ") or "captured_traffic.pcap"
            block_choice = input("Enable automatic IP blocking for suspicious traffic? (y/n): ").lower()
            args.block = True if block_choice == "y" else False
            start_sniffing(interface=args.interface, pcap_file=args.pcap)
        elif mode_choice == "5":
            print("[*] Exiting the Network Utility Tool.")
            break
        else:
            print("[!] Invalid mode selected. Please try again.")
```

This last part just is part of the menu, so the initial menu list calls to this whenever it is being executed. So a lot of the menu face is stored in here.